ЛАБОРАТОРНАЯ РАБОТА №4. API И BLOC (ЧАСТЬ 1)

Цель: освоить работу с API и HTTP-запросами в Flutter, а также научиться управлять состоянием приложения с помощью архитектуры BLoC.

Задачи:

- настроить HTTP-клиент Dio для взаимодействия с внешним API;
- реализовать отправку запроса и обработку полученного ответа (создать модель данных и репозиторий) для первого экрана;
- написать BLoC (Business Logic Component) для управления состоянием первого экрана, интегрировать его и протестировать работу на эмуляторе или физическом устройстве;
- написать отчет о проделанной работе.

ПОЯСНЕНИЯ К ВЫПОЛНЕНИЮ ЗАДАЧ

Установка библиотек

– dio для HTTP-запросов;

– equatable для сравнения объектов;

– flutter_bloc для управления состоянием приложения на основе архитектуры BLoC (Business Logic Component) – разделение бизнес-логики и интерфейса пользователя;

– talker_dio_logger и talker_bloc_logger – дополнения для talker, чтобы просматривать логи dio и bloc;

– build_runner, json_serializable и json_annotation для генерации кода сериализации и десериализации объектов.

Настройка HTTP-клиента

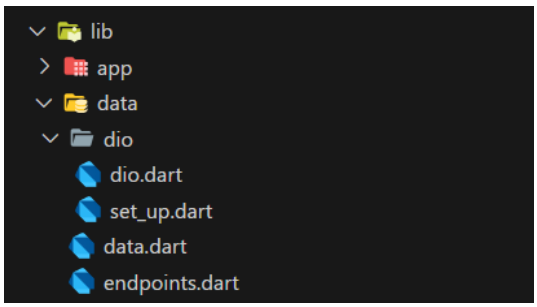
Сперва можно зарегистрировать экземпляр клиента в файле с зависимостями:

di.dart

```
...
final Dio dio = Dio();
...

Future<void> setupLocator() async {
  setUpDio();
  ...
}
```

Функция, отвечающая за саму настройку, будет находиться в директории data/dio/:



set_up.dart

```
void setUpDio() {
  dio.options.baseUrl = ''; // общая часть адресов запросов
  dio.options.queryParameters.addAll({
    'api_token': '', // ключ/токен, выданный при регистрации
  });
  dio.options.connectTimeout = const Duration(seconds: 5);
  dio.options.receiveTimeout = const Duration(seconds: 5);
  dio.interceptors.addAll([
    TalkerDioLogger(
      talker: talker,
      settings: TalkerDioLoggerSettings(
        printRequestData: true,
        printRequestHeaders: true,
      ),
    ),
  ]);
}
```

BaseUrl и api token (или api key) можно найти на сайте используемого API, либо подставить собственные, если API свой.

Здесь же, в папке data, можно увидеть еще один файл – endpoints.dart, в котором будут указаны конечные точки запросов. Их так же можно найти в документации:

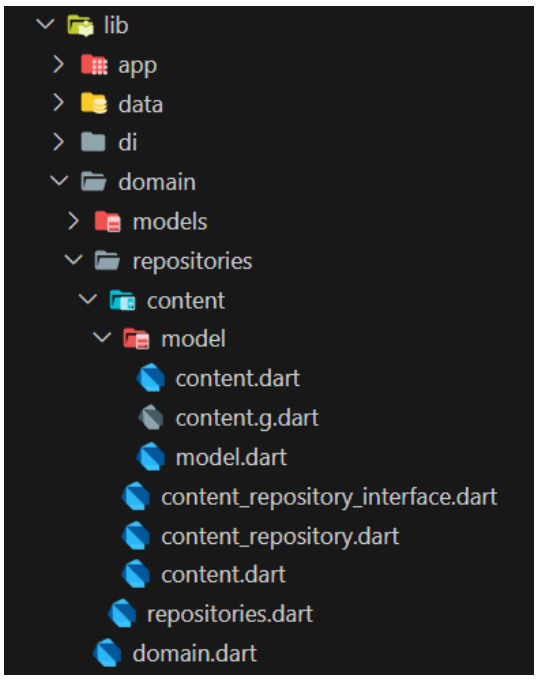
endpoints.dart

```
class Endpoints {
  Endpoints._();

  static const String content = '/content';
}
```

Dart – объектно-ориентированный язык, поэтому данные, описывающие объекты реального мира, собираются в классы, моделирующие эти объекты.

Если класс уникален для какого-либо репозитория, его код обычно хранится внутри его директории, иначе – в глобальной папке всех репозиториев.



content.dart

```
part 'content.g.dart';

@JsonSerializable()
class Content {
  final int id;
  final String title;
  final String author;
  final String description;
  final String image;

  Content({
    required this.id,
    required this.title,
    required this.author,
    required this.description,
    required this.image,
  });

  factory Content.fromJson(Map<String, dynamic> json) =>
      _$ContentFromJson(json);

  Map<String, dynamic> toJson() => _$ContentToJson(this);
}
```

Поля класса должны быть аналогичны полям присылаемого с сервера ответа. Как минимум те, которые нужны в приложении.

Необязательные (которые могут принимать значение null) можно обозначить символом *?*, который указывается после типа. При этом из конструктора нужно убрать ключевое слово *required*.

Если стили написания названий отличаются, аналогия указывается с помощью конструкции *@JsonKey(name: '')* перед полем.

Объявленные методы-фабрики понадобятся для сериализации и десериализации объектов. Инициализация этих методов будет написана в файле с расширением g.dart, который можно автоматически сгенерировать с помощью команды *dart run build_runner build -d*.

Далее можно добавить репозиторий и его интерфейс:

content_repository_interface.dart

```
abstract interface class ContentRepositoryInterface {
  Future<List<Content>> getContent();
}
```

content_repository.dart

```
class ContentRepository implements ContentRepositoryInterface {
  ContentRepository({required this.dio});

  final Dio dio;

  @override
  Future<List<Content>> getContent() async {
    try {
      final Response response = await dio.get(Endpoints.content);
      final content = (response.data['data'] as List)
          .map((e) => Content.fromJson(e))
          .toList();
      return content;
    } on DioException catch (e) {
      throw e.message.toString();
    }
  }
}
```

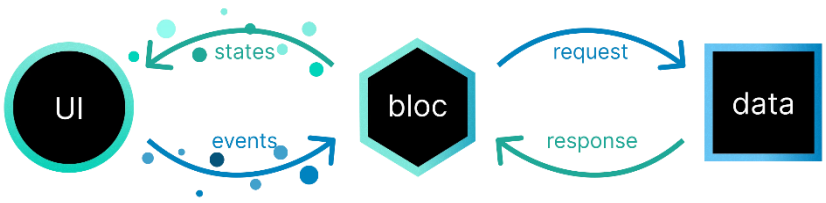
Связь пользовательского интерфейса с репозиториями будет реализована с помощью BLoC (Business Logic Component) – архитектурного паттерна для управления состоянием приложения и логикой его работы. Он включает три основных компонента:

– events – события – действия пользователя или системы, инициируемые UI (например, нажатие кнопки), которые отправляются в блок, чтобы вызвать изменения состояния;

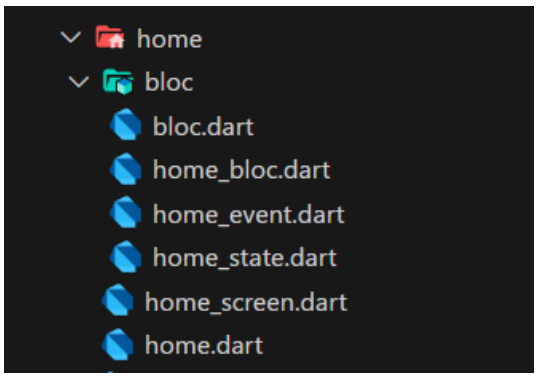
– блок – центральный компонент, который принимает события, выполняет в ответ какую-то логику и отправляет обратно в UI соответствующие состояния;

– states – состояния – определяют, что именно должен видеть пользователь на экране приложения, то есть отвечают за обновление UI.

Таким образом, UI отправляет события в блок, блок обрабатывает их и возвращает состояния, которые обновляют UI.



Для примера домашнего экрана эти компоненты будут выглядеть следующим образом:



home_event.dart

```
part of "home_bloc.dart";

sealed class HomeEvent extends Equatable {
  const HomeEvent();

  @override
  List<Object> get props => [];
}

class HomeLoad extends HomeEvent {
  const HomeLoad({this.completer});

  final Completer? completer;

  @override
  List<Object> get props => [];
}
```

home_state.dart

```
part of "home_bloc.dart";

sealed class HomeState extends Equatable {
  const HomeState();

  @override
  List<Object> get props => [];
}

final class HomeInitial extends HomeState {}

final class HomeLoadInProgress extends HomeState {}

final class HomeLoadSuccess extends HomeState {
  const HomeLoadSuccess({ required this.content});

  final List<Content> content;

  @override
  List<Object> get props => [content];
}
```

```
final class HomeLoadFailure extends HomeState {
  const HomeLoadFailure({required this.exception});

  final Object? exception;

  @override
  List<Object> get props => [];
}
```

home_bloc.dart

```
part "home_event.dart";
part "home_state.dart";

class HomeBloc extends Bloc<HomeEvent, HomeState> {
  final ContentRepositoryInterface contentRepository;

  HomeBloc(this.contentRepository) : super(HomeInitial()) {
    on<HomeLoad>(_homeLoad);
  }

  Future<void> _homeLoad(event, emit) async {
    try {
      if (state is! HomeLoadSuccess) {
        emit(HomeLoadInProgress());
      }
      final content = await contentRepository.getContent();
      emit(HomeLoadSuccess(content: content));
    } catch (exception, state) {
      emit(HomeLoadFailure(exception: exception));
      talker.handle(exception, state);
    } finally {
      event.completer?.complete();
    }
  }

  @override
  void onError(Object error, StackTrace stackTrace) {
    super.onError(error, stackTrace);
    talker.handle(error, stackTrace);
  }
}
```

Далее блок и репозиторий нужно будет зарегистрировать в di перед тем, как непосредственно использовать:

di.dart

```
Future<void> setupLocator() async {
  ...
  getIt.registerSingleton<ContentRepositoryInterface>(
    ContentRepository(dio: dio),
  );
  getIt.registerSingleton(HomeBloc(getIt.get<ContentRepositoryInterface>()));
  ...
}
```

Вызов блока с домашнего экрана будет выглядеть следующим образом:

home_screen.dart

```
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final _home = getIt<HomeBloc>();
  void loadHome() => _home.add(HomeLoad());

  @override
  void initState() {
    loadHome();
    super.initState();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppAppbar(title: Text('Home')),
      body: BlocBuilder<HomeBloc, HomeState>(
        bloc: _home,
        builder: (context, state) {
          return switch (state) {
            HomeInitial() => _buildHomeInitial(),
            HomeLoadInProgress() => _buildHomeLoadInProgress(),
            HomeLoadSuccess() => _buildHomeLoadSuccess(state),
            HomeLoadFailure() => _buildHomeLoadFailure(state),
          };
        },
      ),
    );
  }

  Widget _buildHomeInitial() => SizedBox.shrink();

  Widget _buildHomeLoadInProgress() => AppProgressIndicator();

  Widget _buildHomeLoadSuccess(HomeLoadSuccess state) {
    final content = state.content;

    return SingleChildScrollView(
      padding: EdgeInsets.all(16),
      child: Column(
        spacing: 20,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Header',
            style: Theme.of(context).textTheme.headlineLarge,
          ),
          ListView.separated(
            primary: false,
```

```
            shrinkWrap: true,
            itemCount: content.length,
            itemBuilder: (_, index) => ContentCard(
              content: content[index],
            ),
            separatorBuilder: (_, __) => 16.ph,
          ),
        ],
      ),
    );
  }

  Widget _buildHomeLoadFailure(HomeLoadFailure state) {
    return AppError(
      description: state.exception.toString(),
      onTap: () => loadHome(),
    );
  }
}
```